

Construir la app con Spec Kit

AI Media Generator — 39 tickets en 6 fases, listos para ejecutar.

Repo github.com/sjunka/speckit-ai-generator

Feature 001-ai-media-generator

Fecha 2026-08-18

Qué vamos a hacer

Una app donde el usuario toma una foto, la IA la convierte en una imagen bonita, y esa imagen se vuelve un video corto que puede descargar o compartir.

No vamos a escribir el código a mano. Ya está todo descrito en este repo: qué hace la app, cómo se construye, y en qué orden. El agente de IA lee esa descripción y escribe el código.

Tu trabajo es pasarle los tickets uno por uno y revisar que salga bien.

Qué necesitas

- Claude Code instalado
- Una API key de Anthropic que funcione
- Node 20 o superior
- git

Nada más. No hace falta instalar Spec Kit, ya viene en el repo.

Lista de tareas

- ☐ 1. Clonar el repo
- ☐ 2. Verificar que funciona
- ☐ 3. Leer tres documentos
- ☐ 4. Ticket T001 — crear el proyecto
- ☐ 5. Tickets T002 a T009 — la base
- ☐ 6. Repartir el trabajo entre el equipo
- ☐ 7. Tickets T031 a T039 — juntar todo
- ☐ 8. Desplegar

Son 39 tickets en total. Van del T001 al T039, en orden.

1. Clonar el repo

```
git clone https://github.com/sjunka/speckit-ai-generator.git
cd speckit-ai-generator
```

2. Verificar que funciona

Antes de nada, comprueba que el agente encuentra los archivos:

```
bash .specify/scripts/bash/check-prerequisites.sh --json --require-tasks --include-tasks
```

Si te sale una línea larga que dice `FEATURE_DIR`, todo bien, sigue.

Si dice `Feature directory not found`, falta un archivo. Créalo así y vuelve a probar:

```
printf '{\n  "feature_directory": "specs/001-ai-media-generator"\n}\n' > .specify/feature.json
```

3. Leer tres documentos

Media hora de lectura que ahorra días de trabajo.

1. `docs/PROMPT-NOTES.md` — el más corto. Explica por qué el trabajo está partido en 6 fases y no de otra forma.
2. `specs/001-ai-media-generator/spec.md` — qué hace la app, sin tecnología.
3. `docs/REPLICATION-PROMPT.md`, **sección §10** — 16 errores comunes. Cada uno costó una hora la primera vez. Leerlos es la mejor media hora del proyecto.

El archivo `docs/REPLICATION-APPENDIX.md` **no se lee**. Es la respuesta correcta: tiene los 80 archivos del proyecto original. Sirve para comparar cuando dudas de algo.

4. Primer ticket

```
/speckit-implement T001
```

Esto crea el proyecto de Next.js desde cero. Espera a que termine.

Cómo saber que salió bien: deben existir `package.json`, `app/layout.jsx` y `app/page.jsx`.

5. La base (T002 a T009)

Ocho tickets más, uno por uno:

```
/speckit-implement T002  
/speckit-implement T003
```

...y así hasta el T009. Aquí se construyen los colores, la tipografía, los botones, y la configuración de las pruebas.

Cómo saber que salió bien:

```
npm run lint && npm test && npm run build
```

Los tres tienen que pasar. Cuando pasen, sube esto a `main`.

Nadie puede empezar el siguiente paso hasta que esto esté en `main`.

6. Repartir el trabajo

Aquí es donde tres personas pueden trabajar al mismo tiempo sin estorbarse.

Persona	Tickets	Qué construye
A	T010 a T017	Las pantallas que ve el usuario
B	T018 a T025	El backend y la conexión con la IA
C	T026 a T030	El dashboard del dueño y las cuentas

Cada quien crea su propia carpeta de trabajo:

```
git worktree add ../frontend -b 001-frontend
git worktree add ../backend -b 001-backend
git worktree add ../dashboard -b 001-dashboard
```

Y desde su carpeta arranca con su primer ticket: `/speckit-implement T010`, `/speckit-implement T018` o `/speckit-implement T026`.

¿Por qué no chocan? Porque cada grupo toca archivos completamente distintos. En `tasks.md` está escrito qué archivos son de cada quien y cuáles no puede tocar. Ningún archivo aparece en dos listas.

Un aviso sobre los tickets de C: el T030 es el único que necesita crear cuentas reales (Clerk, MongoDB, Vercel, Higgsfield). Quien lo tome va a tardar más. Los otros dos siguen sin necesitar contraseñas de nada.

Si trabajas solo: olvídate de las carpetas separadas. Haz los tickets en orden, del T010 al T030. Funciona igual, solo que más lento.

7. Juntar todo (T031 a T039)

```
/speckit-implement T031
```

...hasta el T039. Aquí se juntan las tres ramas y se arregla lo que se rompió.

El ticket importante de esta parte es el T032. Mientras trabajaban por separado, la persona A probó su código contra un backend falso. Ahora hay que comprobar que el backend de verdad se comporta igual. Si no coinciden, las pruebas pasan pero la app no funciona. No te saltes ese ticket.

8. Desplegar

El T039 lo explica. Vercel se conecta al repo y despliega solo. Solo hay que poner las 8 variables de entorno en la configuración del proyecto.

Si algo falla

Qué ves	Qué hacer
Feature directory not found	Falta <code>.specify/feature.json</code> . Ver el paso 2
El comando <code>/speckit-implement</code> no existe	Estás fuera de la carpeta del repo, o usas otro agente que no es Claude Code
El agente reescribe <code>tasks.md</code>	Alguien corrió <code>/speckit-tasks</code> . Reviértelo. Ese archivo no se regenera nunca
Un botón sale sin estilos	Error 16 de la sección §10
Una clase de Tailwind no hace nada	Error 15 de la sección §10. Tailwind no avisa cuando te equivocas de nombre
La imagen generada no aparece	Error 13 de la sección §10
El agente se inventa detalles	No leyó las secciones §6 y §7. Pásaselas a mano

Seis reglas que no se rompen

1. **La prueba primero.** Se escribe la prueba que falla, después el código que la arregla. En ese orden.
2. **Todo funciona sin internet.** Ninguna prueba necesita contraseñas ni red.
3. **Solo se construye lo que está en la lista.** Si no está en el spec, no se hace. Nada "por si acaso".
4. **Los colores viven en un solo archivo.** `app/globals.css` . Un color escrito en cualquier otro lado es un error.
5. **Cada quien toca sus archivos.** Si necesitas cambiar uno que no es tuyo, lo avisas, no lo cambias.

6. **Si hay dudas, gana el apéndice.** `REPLICATION-APPENDIX.md` tiene la versión correcta de cada archivo.

Están completas en `.specify/memory/constitution.md`.

Antes de la presentación

- ☐ Todos pueden clonar el repo
- ☐ El comando del paso 2 funciona en la máquina de cada uno
- ☐ **Alguien ya corrió el T001 completo y funcionó**
- ☐ Todos tienen Claude Code y una API key
- ☐ Todos leyeron la sección §10

El tercer punto es el único que no se puede improvisar en vivo. Háganlo antes.

Si necesitas cambiar algo del plan

Los archivos de `specs/` mandan. Los editas, los subes, y avisas al equipo.

Lo único que **no** debes hacer es regenerar `tasks.md` con `/speckit-tasks`. Ese archivo dice quién es dueño de cada archivo, y eso es lo que permite que tres personas trabajen a la vez. Si lo regeneras, se pierde. Si necesitas tickets nuevos, escríbelos a mano copiando el formato de los que ya están.